

Arrays

1. Arrays

- An **array** is a group of consecutive memory locations with the same name and the same type.
- Simple variable is a single memory location with a unique name and type, but an array is a collection of different consecutive/adjacent memory locations. All these memory locations have one collective name and type.
- The memory locations in the array are known as **elements** of array.
- The total elements in the array is called its **length**.
- Each element in the array is accessed with reference to its position of location in the array. This position is called **index** or **subscript**.

1. Arrays (continued...)

- Each element in the array has a unique index. The index of first element is *0* and the index of last element is *length-1*. The value of the index is written in brackets along with the name of array.
- Arrays are used *to store a large amount of similar kind of data*.
- Suppose the user wants to store marks of 100 students and declares 100 variables. It is time-consuming process to use these variables individually. This process can be simplified by using array. *An array of 100 elements can be declared to store these values instead of 100 individual variables. (ADVANTAGE)*

1.1 Advantages / Uses of Arrays

Some advantages of arrays are as follows:

- Arrays can store a large number of values with single name and single type.
- Arrays are used to process many values easily and quickly.
- The values stored in an array can be sorted easily.
- A search process can be applied on arrays easily.

1.2 Declaring One-Dimensional Array

- A type of array in which all elements are arranged in the form of a list is known as ***one-dimensional*** array.
- It is also called ***single-dimensional array*** or ***linear list***.
- It consists of one column or one row. The process of specifying array name, length and data type is called ***array declaration***.

1.2 Declaring One-Dimensional Array (continued...)

Syntax

The syntax of declaring one-dimensional array is as follows:

Data_Type Identifier[Length];

Data_Type : It indicates the data types of the values to be stored in the array.

Identifier : It indicates the name of the array.

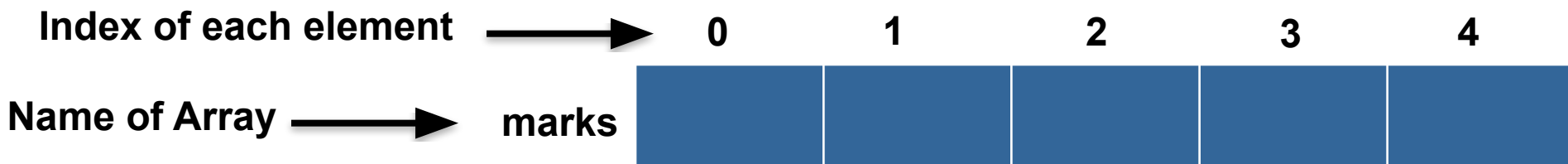
Length : It indicates total number of elements (memory locations) in the array.

1.2 Declaring One-Dimensional Array (continued...)

Example

```
int marks[5];
```

The above example declares an integer (**data type**) array **marks** (**identifier/name**) of five (**length**) elements. It allocates five consecutive locations in memory. The index of first element is 0 and the index of last element is : $\text{Length} - 1 = 5 - 1 = 4$.



1.3 Array Initialization

- The process of assigning values to array elements at the time of array declaration is called ***array initialization***.
- The initialization process provides a list of initial values for array elements. The values are separated with commas and enclosed within braces.
- A syntax error occurs if the values in braces are more than the length of array.
- If the number of initial values is less than the array size, the remaining array elements are initialized to zero.

1.3 Array Initialization (continued...)

Syntax

The syntax to initialize array is as follows:

Data_Type Identifier[Length] = {List of values};

Data_Type : It indicates the data types of the values to be stored in the array.

Identifier : It indicates the name of the array.

Length : It indicates total number of elements (memory locations) in the array.

List of Values : It indicates the values to initialize the array. These values must be constant.

1.3 Array Initialization (continued...)

Example

```
int marks[5] = {70, 54, 82, 96, 49};
```

The above example declares an integer array *marks* with five elements. It also initializes the array with values given in the braces. In this declaration, marks[0] is initialized to 70 marks, marks[1] is initialized to 54 and so on.

Index of each element	→	0	1	2	3	4	
Name of Array	→	marks	70	54	82	96	49

1.3 Array Initialization (continued...)

An array size can be omitted when it is initialized in the declaration as follows:

```
int age[ ] = {23, 56, 87, 92, 38, 12, 15, 6, 3};
```

The compiler determines the size of the **age** array according to the initialized values on right side. The size of array is 9.

1.4 Accessing Individual Elements of Array

Each individual element of an array is accessed by specifying the following:

- Name of array
- Index of element

1.4 Accessing Individual Elements of Array (continued...)

Syntax

The syntax of accessing an individual element of an array is as follows:

Array_Name[Index];

Array_Name : It indicates the name of array.

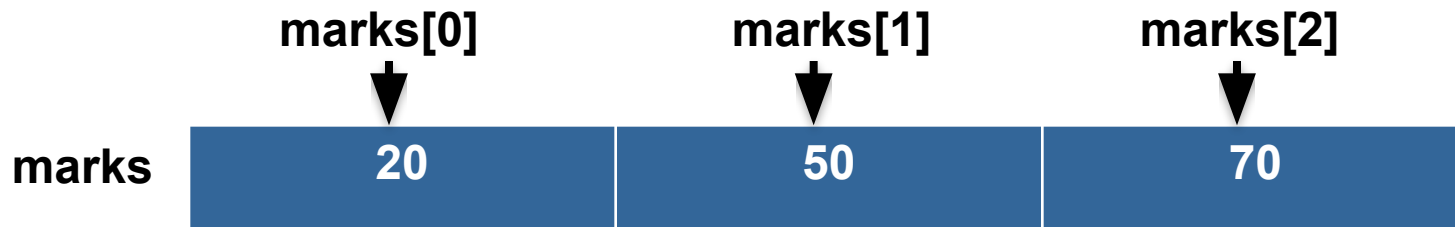
Index : It indicates the index of element to be accessed.

1.4 Accessing Individual Elements of Array (continued...)

Example

```
int marks[3];  
marks[0] = 20;  
marks[1] = 50;  
marks[2] = 70;
```

The above example declares an array *marks* of type *int* with three elements. It accesses all elements of the array individually to store different values. *marks[0]* indicates the first element and *marks[2]* indicates the last element of the array.



1.5 Accessing Array Elements using Loops

- An *easier and faster way of accessing array elements is using loops*. The following example shows how array elements can be accessed using **for** loop.

```
#include<iostream>
using namespace std;
int main()
{
    int marks[5] = {10, 20, 30, 40, 50};
    for(int i=0;i<5;i++)
    {
        cout<<marks[i]<<endl;
    }
    return 0;
}
```

Output

```
10
20
30
40
50
-----
Process exited after 0.07279 seconds with return value 0
Press any key to continue . . .
```

1.6 Input and Output Values of an Array

- The process of input and output with arrays is similar to the input and output with simple variables. The **cin** object is used to *input values* in the arrays. The **cout** object is used to *display values* of arrays. However, *the use of these objects with each individual elements becomes very time-consuming* because each element is treated as a simple variable.
- **Loops** are frequently used to input and output data in an array. *The use of loops with arrays makes the process of input and output faster and easier.* It also reduces the code written for this purpose.

1.6 Input and Output Values of an Array (continued...)

Prog 1: Write a program that inputs five integers from the user and stores them in an array. It then displays all values in the array. (Do not use loops)

```
#include<iostream>
using namespace std;
int main()
{
    int arr[5];
    cout<<"Enter five integers:"<<endl;
    cin>>arr[0];
    cin>>arr[1];
    cin>>arr[2];
    cin>>arr[3];
    cin>>arr[4];
    cout<<"The values in array are":<<endl;
    cout<<arr[0]<<endl;
    cout<<arr[1]<<endl;
    cout<<arr[2]<<endl;
    cout<<arr[3]<<endl;
    cout<<arr[4]<<endl;
    return 0; }
```

Output

```
Enter five integers:
10
20
30
40
50
The values in array are:
10
20
30
40
50
-----
Process exited after 7.939 seconds with return value 0
Press any key to continue . . .
```

1.6 Input and Output Values of an Array (continued...)

Prog 2: Write a program that inputs five integers from the user and stores them in an array. It then displays all values in the array **using loops**.

```
#include<iostream>
using namespace std;
int main()
{
    int arr[5];
    for(int i=0;i<5;i++)
    {
        cout<<"Enter an integer: ";
        cin>>arr[i];
    }

    cout<<"The values in array are:"<<endl;

    for(int i=0;i<5;i++)
    {
        cout<<arr[i]<<endl;
    }
    return 0;
}
```

Output

```
Enter an integer: 10
Enter an integer: 20
Enter an integer: 30
Enter an integer: 40
Enter an integer: 50
The values in array are:
10
20
30
40
50

-----
Process exited after 12.65 seconds with return value 0
Press any key to continue . . .
```

1.6 Input and Output Values of an Array (continued...)

Prog 3: Write a program that inputs five values from the user and stores them in an array and then displays the sum and average of these values.

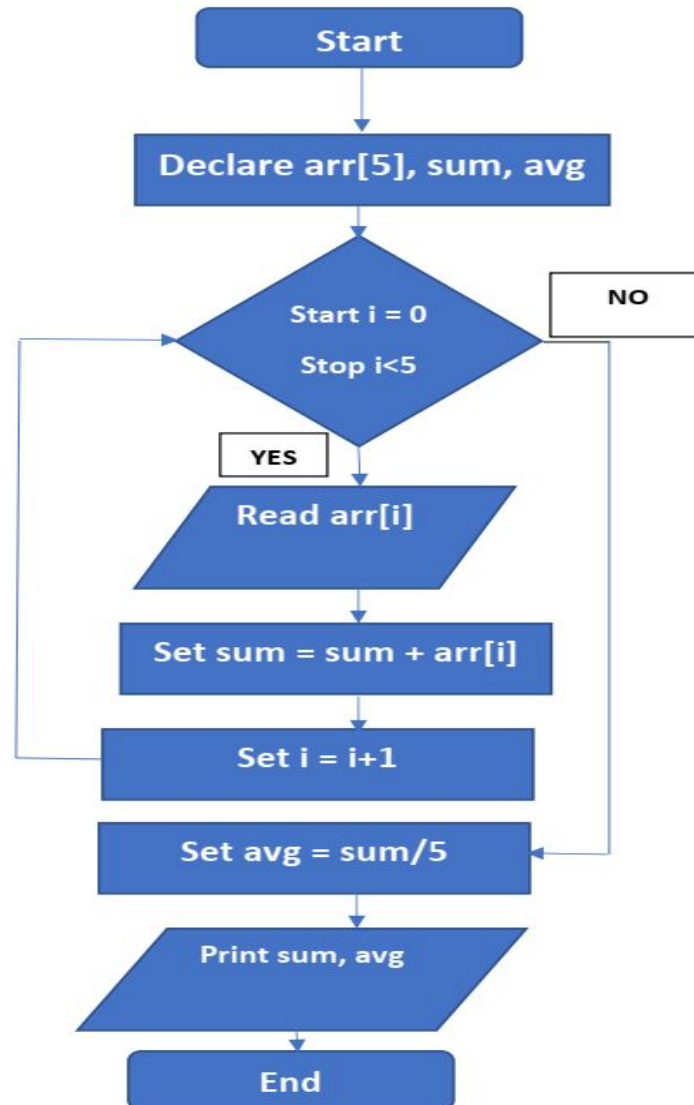
```
#include<iostream>
using namespace std;
int main()
{
    int arr[5], sum = 0;
    float avg = 0.0;
    for(int i=0;i<5;i++)
    {
        cout<<"Enter value:";
        cin>>arr[i];
        sum = sum + arr[i];
    }
    avg = sum/5.0;
    cout<<"Sum is: "<<sum<<endl;
    cout<<"Average is: "<<avg;
    return 0;
}
```

Output

```
Enter value:10
Enter value:72
Enter value:13
Enter value:17
Enter value:60
Sum is: 172
Average is: 34.4
-----
Process exited after 30.89 seconds with return value 0
Press any key to continue . . .
```

**Flow chart and working
of this program is in next
slide(s)**

1.6 Input and Output Values of an Array (continued...)



Flow chart

1.6 Input and Output Values of an Array (continued...)

- Suppose the user enters the following values in the array in **program no 3** i.e., {3, 7, 5, 9, 6}.

Iteration No.	Value of i	Statement Executed	Value of Sum
1 st	0	Sum = sum+arr[0] Sum = 0+3	3
2 nd	1	Sum = sum+arr[1] Sum = 7+3	10
3 rd	2	Sum = sum+arr[2] Sum = 5+10	15
4 th	3	Sum = sum+arr[3] Sum = 15+9	24
5 th	4	Sum = sum+arr[4] Sum = 24+6	30

1.6 Input and Output Values of an Array (continued...)

Home Work Assignment

PROGRAM 4: Write a program that inputs ten numbers from the user in an array and displays the maximum number.

PROGRAM 5: Write a program that inputs ten numbers from the user in an array and displays the minimum number.

1.7 Searching in Arrays

Searching is a process of *finding the required data in the array*. Searching *becomes more important* when the *length of the array is very large*.

Sequential Search- is also called **Linear search** or **serial search**. It is a simple way to search an array for the desired value. It follows the following steps to search a value in array.

- Visit the first element of the array and compare its value with the required value.
- If the value of array matches with the desired value, the search is complete.
- If the value of array doesn't match, move to next element and repeat same process.

Loops are frequently used to visit elements of array for searching a value. You can start the counter variable of loop from 0 and move it to last index of array.

1.7 Searching in Arrays (continued...)

SEARCHING IN ARRAYS

Linear Search

Algorithm

1. Set counter (ctr) to lower bound ($L = L.B$)
i.e, $ctr = L.B = 0$
2. Repeat Steps 3 through 4 until
 $ctr > U$ [$U = U.B = \text{upper bound}$]
3. If array counter ($Ar[ctr]$) is
equal to required item then
print "Element found", location of element & exit from loop
If $Ar[ctr] == \text{item}$ then
Print "Element found"
Print ctr : Break;
4. Increment the counter by 1 i.e, $ctr = ctr + 1$
5. If $ctr > U$, then print "Element not found"
6. END.

1.7 Searching in Arrays (continued...)

Working Example of Linear Search

Let Array = A

1	5	2	4	6	8
0	1	2	3	4	5

Let's say if we want to find location of element 4.

1- Set counter to ZERO i.e. $ctr = 0$

2- If:

i) $A[0] == 4$; $1 == 4$ [FALSE]

Increment the counter by one i.e., $ctr = 0 + 1 = 1$

ii) $A[1] == 4$; $5 == 4$ [FALSE]

Increment the counter by one i.e., $ctr = 1 + 1 = 2$

iii) $A[2] == 4$; $2 == 4$ [FALSE]

Increment the counter by one i.e., $ctr = 2 + 1 = 3$

iv) $A[3] == 4$; $4 == 4$ [TRUE]

1.7 Searching in Arrays (continued...)

Element Found at Index 3 and Exit (break)
from loop and Jump to END state.

→ Let's say if we want to find location of ^{an} element which is not given in the array. The element will be searched till index 5 & the next time when counter (ctr) is incremented, its value will be 6 i.e. $ctr = 6$ & step 5 states If $ctr > 4 = 6 > 5$ print "Element Not Found" & END.

1.7 Searching in Arrays (continued...)

Disadvantage of Linear Search

Suppose we want to locate element 8 (in previous example), all elements before the required element will be searched, which is a time consuming task.

1.7 Searching in Arrays (continued...)

Linear Search Code:

```
#include<iostream>

using namespace std;

int main()
{
    int arr[10], item, index, found=0;
    cout<< "Enter the Elements of Array:"<<endl;
    for(int i=0;i<10;i++)
    {cin>>arr[i]; }
    cout<<"The Array elements are as follows: "<<endl;
    for(int i=0;i<10;i++)
    {cout<<arr[i]<<endl; }
```

1.7 Searching in Arrays (continued...)

```
cout<<"Enter the element to search:"<<endl;
cin>>item;
for(int i=0;i<10;i++)
{
    if(arr[i]==item)
    {
        index = i;
        found = 1;
        break;
    }
}
cout<<"The Element: " << item << " is found at index:" << index <<endl;
if(found==0)
{cout<<"The Element: " << item <<" is not found in the array:"<<endl;}
return 0;
}
```

1.7 Searching in Arrays (continued...)

Output

```
Enter the Elements of Array:
12
54
77
89
190
59
29
43
67
91
The Array elements are as follows:
12
54
77
89
190
59
29
43
67
91
Enter the element to search:
29
The Element: 29 is found at index:6
-----
Process exited after 68.99 seconds with return value 0
Press any key to continue . . .
```

1.7 Searching in Arrays (continued...)

Binary Search- is a *quicker method* of searching for value in the array. *It takes less time than sequential search.*

- It locates the middle element of array and compares with the search number.
- If they are equal, search is successful and the index of middle element is returned.
- If they are not equal, reduces the search to half of the array.
- If the search number is less than the middle element, the first half of the array is searched otherwise the second half of the array is searched.
- The process continues until the required number is found or loop completes without successful search.

Binary search is very quick but it can search an array only if it is sorted. It cannot be applied on an unsorted array.

1.7 Searching in Arrays (continued...)

Binary Search

Algorithm

- Array must be sorted.
- we have to declare two variables Low & High
- in C++ ; $Low = 0$ & $High = Arraysize - 1$.
- while loop will be executed as long as low is equal to or less than high.

→ while ($low \leq high$)

do $mid \leftarrow (low + high) / 2$

if $V = A[mid]$
return mid

$V = \text{value of item to be searched}$

1.7 Searching in Arrays (continued...)

if $V > A[mid]$
 $low \leftarrow mid + 1$

else
 $high \leftarrow mid - 1$
return

Working Example of Binary Search

Let Array $A =$

10	14	19	26	31	42	44
0	1	2	3	4	5	6

Find location of Value 14.

→ As we can see the array is sorted in ascending order.

→ Here $V = 14$, $Low = 0$ & $High = 6$

1.7 Searching in Arrays (continued...)

So $mid = (low + high) / 2 = (0 + 6) / 2 = 3$

$mid = 3$

Now if:

i- $V = A[mid]$; $14 == A[3]$; $14 == 26$ [FALSE]
ii- $V > A[mid]$; $14 > A[3]$; $14 > 26$ [FALSE]

else

$high = mid - 1$; $= 3 - 1 = 2$

Now the Array has reduced to :

10	14	19
0	1	2

So again

$mid = (low + high) / 2 = \frac{0 + 2}{2} = \frac{2}{2} = 1$

$mid = 1$

1.7 Searching in Arrays (continued...)

Now if:

$i - v == A[mid]$; $14 == A[1]$; $14 == 14$ [TRUE]

and return mid [return 1].

Home Work Assignment

i) Find the location of 42 of the given array

A =	10	14	19	26	31	42	44	50
	0	1	2	3	4	5	6	7

ii) Write an algorithm for part i).

1.7 Searching in Arrays (continued...)

Binary Search Code:

```
#include<iostream>

using namespace std;

int main()
{
    int arr[10], item, low, high, index, mid, found=0;

    cout<< "Enter the Elements of Array:"<<endl;

    for(int i=0;i<10;i++)
    {cin>>arr[i]; }

    cout<<"The Array elements are as follows: "<<endl;

    for(int i=0;i<10;i++)
    {cout<<arr[i]<<endl; }
```

1.7 Searching in Arrays (continued...)

```
cout<<"Enter the element to search:"<<endl;
cin>>item;
while(low<= high)
{
    mid =(low+high)/2;
    if(item==arr[mid])
    {
        index=mid;
        found=1;
        break;
    }
    else if(item>arr[mid])
        low = mid+1;
    else
        high=mid-1;
}
```

1.7 Searching in Arrays (continued...)

```
cout<<" The elements: "<< item << " is found at index: "<< index<<endl;
if(found==0)
{
cout<<"The Element: " << item <<" is not found in the array:"<<endl;
}
return 0;
}
```

1.7 Searching in Arrays (continued...)

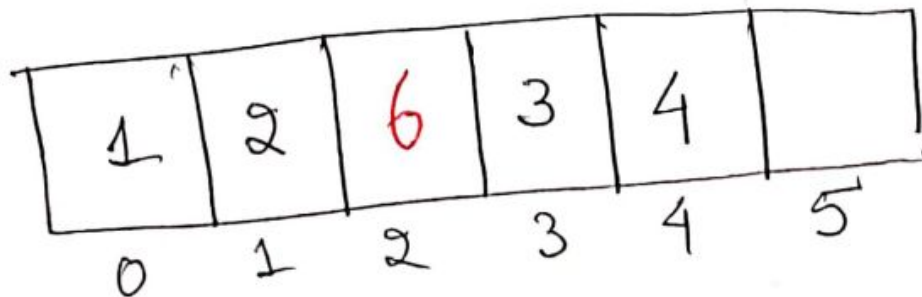
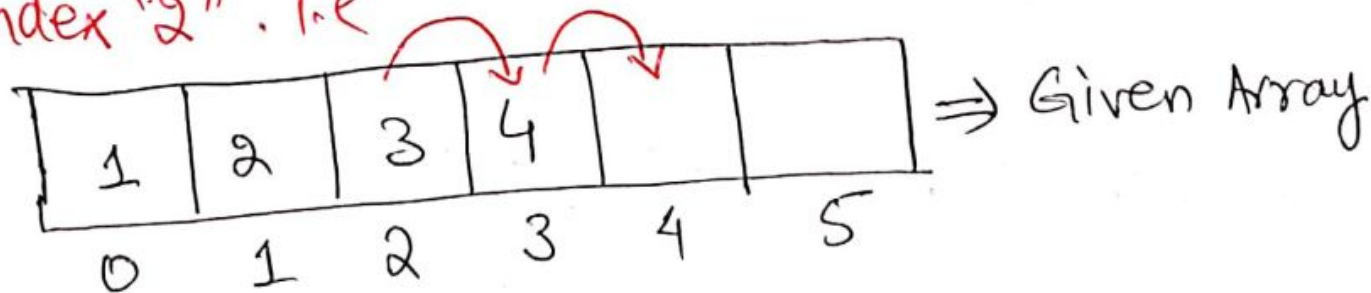
Output

```
Enter the Elements of Array:
10
20
30
40
50
60
70
80
90
100
The Array elements are as follows:
10
20
30
40
50
60
70
80
90
100
Enter the element to search:
90
The Element: 90 is found at index:8
-----
Process exited after 21.94 seconds with return value 0
Press any key to continue . . .
```

1.8 Insertion in Arrays

Inserting element at specified position [Unsorted Array]

Let's say we have an array of size "6" and we want to insert a new value "6" at position/index "2". i.e



1.8 Insertion in Arrays (continued...)

Enter New Element at Specific Position

```
#include<iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
int arr[50], size, no, i, pos;
```

```
cout<<"Enter Array Size: "<<endl;
```

```
cin>>size; //4 (let's say user input array size "4". 0,1,2,3)
```

```
cout<<"Enter array elements: "<<endl;
```

```
for(i=0;i<size;i++)
```

```
{
```

```
cin>>arr[i];
```

```
}
```

Let's say user enters these four elements

1	2	3	4
---	---	---	---

arr[0] arr[1] arr[2] arr[3] -----

1.8 Insertion in Arrays (continued...)

```
cout<<"Enter element to be inserted: "<<endl;
```

```
cin>>no; // 6 , no = 6
```

```
cout<<"Enter the index at which you want to insert a new number:"<<endl;
```

```
cin>>pos; // 2 , pos = 2
```

```
if(pos<size) // 2 < 4 (True)
```

```
{
```

```
for(i=size;i>pos;i--) // i=4; i>2; i--
```

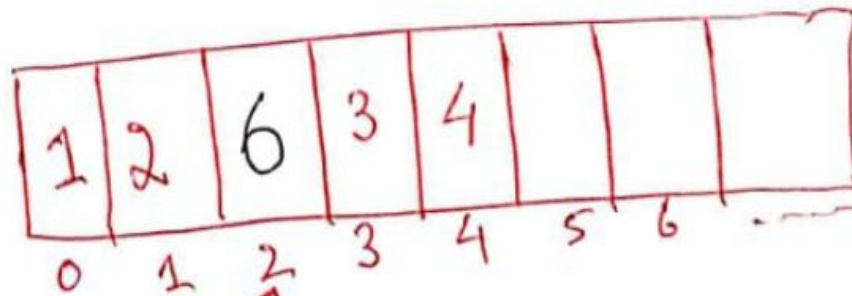
```
{
```

```
arr[i]=arr[i-1]; // 1st iteration arr[4]=arr[3] / 2nd iteration arr[3]=arr[2]
```

```
}
```

```
arr[pos]=no;
```

```
}
```



arr[2]=6

1.8 Insertion in Arrays (continued...)

```
else
{
arr[pos]=no;
}

cout<<"Element inserted successfully...!!"<<endl;

cout<<"New array after insertion of new number is:"<<endl;

for(i=0;i<size+1;i++)
{
cout<<arr[i]<<" ";
}

return 0;
}
```

This part will execute if $pos > size$ say user input $pos = 5$; then there won't be any need of shifting numbers i.e. $arr[5] = 6$

1.8 Insertion in Arrays (continued...)

Output

```
Enter Array Size:
4
Enter array elements:
1
2
3
4
Enter element to be inserted:
6
Enter the index at which you want to insert a new number:
2
Element inserted successfully...!!
New array after insertion of new number is:
1 2 6 3 4
-----
Process exited after 9.893 seconds with return value 0
Press any key to continue . . .
```

1.8 Insertion in Arrays (continued...)

Inserting Element at appropriate position [sorted Array]

→ Let's say we entered the values in array given below

1	3	4	6		
---	---	---	---	--	--

→ Let's say we want to new number/new value "2"
BUT the problem is we don't know the position or
index at first and it has to be found.
Position to be found = ?

→ Remember the given array is sorted in ascending
order & we know ^{new} number/new value "2" will
be added b/w numbers 1 & 3. But how to do
it? (we will see it in a code)

HOME ASSIGNMENT

Implement this logic
using array in C++

1.8 Insertion in Arrays (continued...)

- Secondly it is obvious that a new number is added b/w two numbers or b/w two indexes, so we have to compare the new number from a number at previous index and from a number at next index.
- In unsorted arrays we get input about the position of new value which had to be inserted.
- In sorted arrays we find out position of new number by calling a function.
- Code is very much similar to code of unsorted array except few additions.

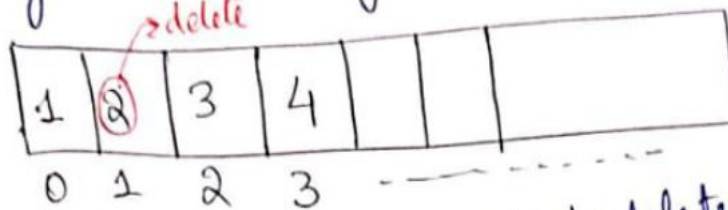
1.9 Deletion in Arrays

Deletion in Arrays

Deletion in arrays

→ Element to be deleted is first searched in array using linear/binary search.

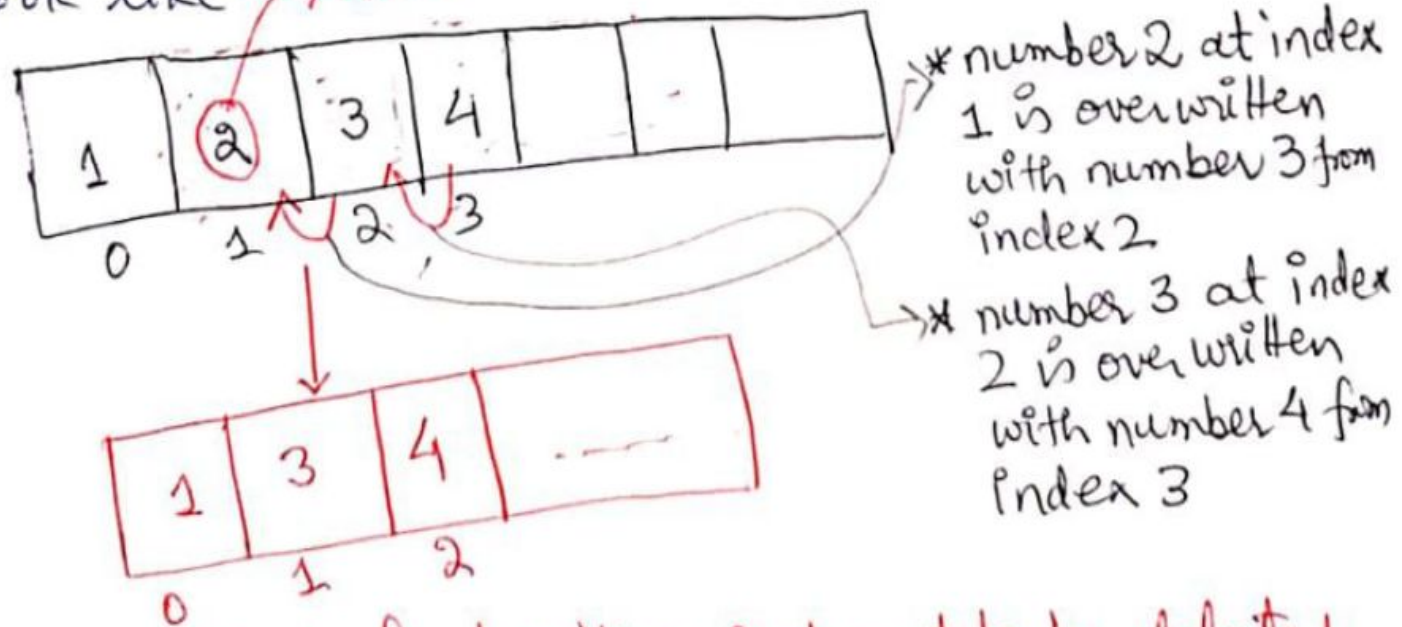
* Let's say
Element to be
deleted = 2.



→ In array when we are going to delete a certain number, (in this case 2), we will overwrite with some other thing ^{current} OR we have to decrease the size of the array. If the above array is used to store 4 numbers of elements, deletion operation will reduce it to store 3 number of elements

1.9 Deletion in Arrays (continued...)

→ After deletion of number "2" the above array will look like



→ Main Task is to find position of element to be deleted and then shifting remaining elements.

1.9 Deletion in Arrays (continued...)

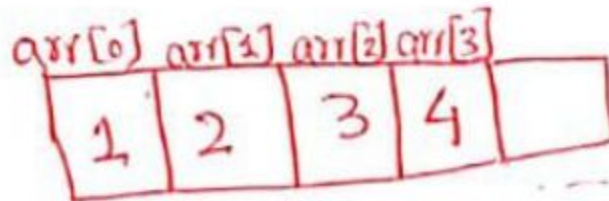
Deletion in Arrays code:

```
#include<iostream>

using namespace std;

int main()
{
    int arr[50], size, no, i, pos, found=0;

    cout<<"Enter Array Size: "<<endl;
    cin>>size; // size = 4
    cout<<"Enter array elements: "<<endl;
    for(i=0;i<size;i++)
    {
        cin>>arr[i];
    }
```



1.9 Deletion in Arrays (continued...)

```
cout<<"Enter element to be deleted: "<<endl;
```

```
cin>>no; // no=2
```

```
for(i=0;i<size;i++)
```

```
{
```

```
if(arr[i]==no)
```

1st iteration

arr[0]==no; 1==2 (false)

2nd iteration

arr[1]==no; 2==2 (True)

```
{
```

```
for(int j=i;j<(size-1);j++)
```

// j=1; j<3; j++

j=2; j<3; j++

```
{
```

arr[j]=arr[j+1]; // arr[1]=arr[2]

arr[2]=arr[3]

```
}
```

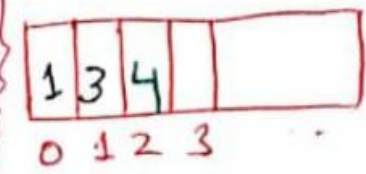
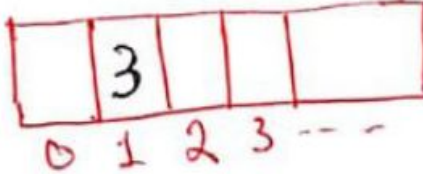
```
found++;
```

```
break;
```

```
}
```

```
}
```

This loop will execute only two times



found = 1

1.9 Deletion in Arrays (continued...)

```
if(found==0)
{
    cout<<"The Element: "<<no<<"is not found in the array:" <<endl;
}
```

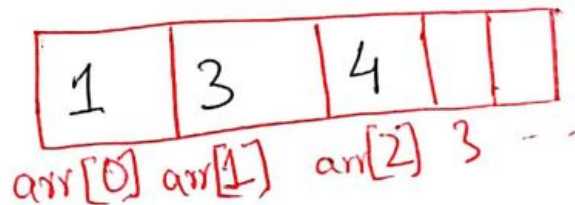
else

```
{
    cout<<"Element is deleted successfully...!!"<<endl;
    cout<<"Array after deletion is:"<<endl;
```

```
for(i=0;i<(size-1);i++)
```

// $i=0; i<3; i++$ [Loop will execute from $i=0$ to $i=2$]

```
{
    cout<<arr[i]<<" ";
}
}
```



```
return 0;
}
```

1.9 Deletion in Arrays (continued...)

OUTPUT

```
Enter Array Size:
4
Enter array elements:
1
2
3
4
Enter element to be deleted:
2
Element is deleted successfully...!!
Array after deletion is:
1 3 4
-----
Process exited after 17.23 seconds with return value 0
Press any key to continue . . .
```

2 Arrays Traversal

Traversal in Arrays

- > It is the processing of all elements one by one.
- Suppose we have an array "arr" given below.

array = arr =

1	2	3	4	5	6
---	---	---	---	---	---

for (int i=0; i<=5; i++)

{

cout << arr[i];

}

output

arr[0] = 1

arr[1] = 2

⋮

arr[5] = 6

2 Arrays Traversal (continued...)

```
#include<iostream>
using namespace std;
int main()
{
    int arr[50], size, no, i;
    cout<<"Enter Array Size: "<<endl;
    cin>>size;
    cout<<"Enter array elements: "<<endl;
    for(i=0;i<size;i++)
    {
        cin>>arr[i];
    }
    cout<<"Printing Array Elements: "<<endl;
    for(i=0;i<size;i++)
    {
        cout<<arr[i]<<" ";
    }
    return 0;
}
```

Output

```
Enter Array Size:
4
Enter array elements:
1
2
3
4
Printing Array Elements:
1 2 3 4
-----
Process exited after 6.716 seconds with return value 0
Press any key to continue . . .
```

3 Sorting in Arrays

Selection Sort is a process of repeatedly select smallest key in the remaining unsorted array.

Algorithm

Small = arr[L] : L1

for i=L to U do : L2

{ : L3

! Small = arr[i], pos=i : L4

for j=i+1 to U do : L5

{ : L6

if arr[j] < Small then : L7

{ : L8

Small = arr[j] : L9

pos = j : L10

} : L11

{ : L12

j = j + 1 : L13

temp = arr[i] : L14

arr[i] = Small : L15

arr[pos] = temp : L16

} : L17

End : L18

3 Sorting in Arrays (continued...)

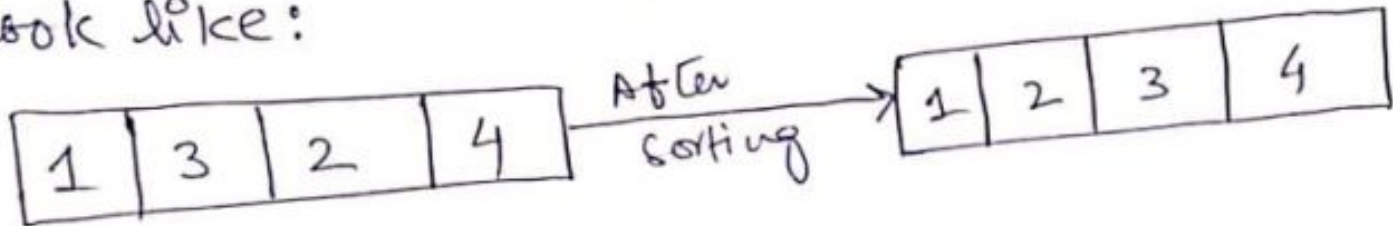
→ Let's say we have an array with elements entered as:

1	3	2	4
---	---	---	---

(19)

→ The array is not sorted i.e (not in ascending or descending order)

→ If we sort the array in ascending order it will look like:



3 Sorting in Arrays (continued...)

Working Example of Selection Sort

→ Given array in unsorted form.

15	6	13	22	3	52	2
0	1	2	3	4	5	6

index:

$L = \text{lower bound} = 0$

$U = \text{Upper bound} = 6$

→ According to our algorithm we will declare a variable "small" and initialize it as: $\text{small} = \text{arr}[L] = \text{arr}[0] = 15$

→ After this the loop will be executed from 0 to 6. (L2)

→ position = pos = $i = 0$.

→ For $j = i + 1$ to U ; that means we will compare the value at index 0, that is 15 with all the elements placed on index 1, 2, 3, 4, 5 & 6 and find out the smallest value. the loop will execute for $j = i + 1$ to U ; $j = 0 + 1$ to 6; $j = 1$ to 6 (L5)

3 Sorting in Arrays (continued...)

- if $\text{arr}[j] < \text{small}$. i.e, $\text{arr}[1] = 6$ & $\text{small} = 15$,
 $6 < 15 \rightarrow \text{TRUE/YES}$ then $\text{small} = \text{arr}[j]$; $\text{small} = 6$,
 $\text{pos} = j$; $\text{pos} = 1$
- Now increment " j " by one. i.e, $j = j + 1$; $j = 1 + 1$; $j = 2$
For $j = 2$
- * if $\text{arr}[j] < \text{small}$.
→ $\text{arr}[2] < \text{small}$
→ $13 < 6$ (FALSE)
- Increment " j " by one, i.e $j = 2 + 1 = j = 3$
- * if $\text{arr}[j] < \text{small}$
→ $\text{arr}[3] < \text{small}$
→ $22 < \text{small}$ (FALSE)

3 Sorting in Arrays (continued...)

→ Increment "j" by one, i.e. $j = 3 + 1$

* if $\text{arr}[4] < \text{small}$

→ $\text{arr}[4] < \text{small}$

→ $4 < 6$ (TRUE)

then $\text{small} = \text{arr}[j]$; $\text{small} = \text{arr}[4] = 4$

$\text{small} = 4$, $\text{pos} = j$, $\text{pos} = 4$

→ Now increment "j" by one, i.e. $j = 4 + 1 = j = 5$

* if $\text{arr}[5] < \text{small}$

→ $\text{arr}[5] < \text{small}$

→ $5.2 < 4$ (FALSE)

3 Sorting in Arrays (continued...)

→ For. $j = 6$

(2)

if $arr[j] < small$

$arr[6] < small$

$2 < 4$ (TRUE)

$small = arr[j]$; $small = arr[6]$; $small = 2$; $pos = j = 6$

→ Small element found is: $small = 2$; now we need to interchange the value as:

$temp = arr[i]$; 14

$temp = arr[0]$

$temp = 15$

3 Sorting in Arrays (continued...)

$arr[pos] = \text{small} : L15$

$arr[0] = \text{small}$

$arr[0] = 2$

$arr[pos] = \text{temp} : L16$

$arr[6] = \text{temp}$

$arr[6] = 15$

→ For the 1st time when $i=0$,
was executed we get

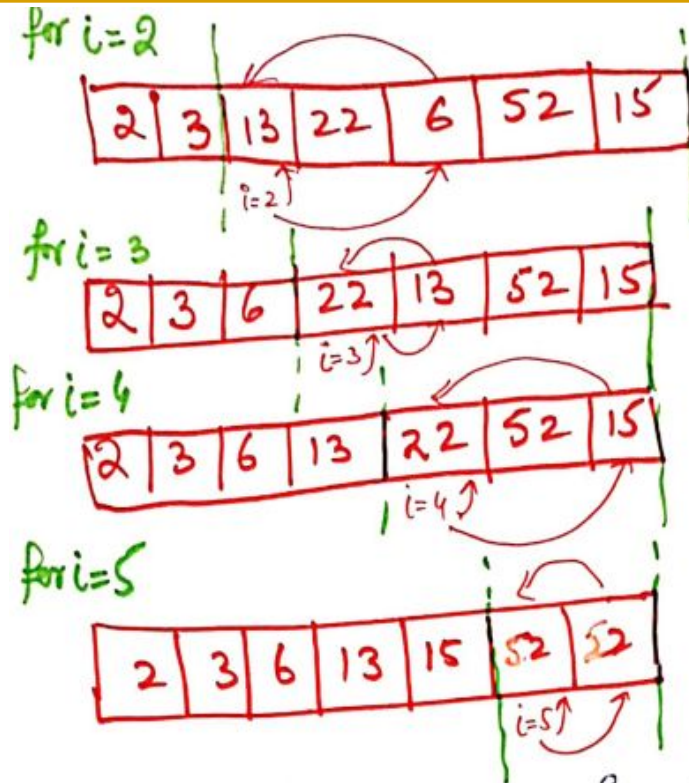
2	6	13	22	3	52	15
---	---	----	----	---	----	----

→ when i is incremented by 1, i.e., $i = i + 1 = 0 + 1 = 1$

2	6	13	22	3	52	15
---	---	----	----	---	----	----

(Diagram showing index i moving from 0 to 1 and a comparison between arr[0]=2 and arr[1]=6)

3 Sorting in Arrays (continued...)



22

The final sorted array in ascending order will be:

2	3	6	13	15	22	52
---	---	---	----	----	----	----

3 Sorting in Arrays (continued...)

Selection Sort code

```
#include<iostream>
using namespace std;
int main()
{
    int i, j, size, pos, temp, small, arr[30];
    cout<<"Enter size of the array: ";
    cin>>size; //4
    cout<<endl;
    cout<<"Enter the elements"<<endl;
    for (i=0;i<size;i++)
    {
        cin>>arr[i];
    }
```

1	3	2	4
0	1	2	3

3 Sorting in Arrays (continued...)

```
for(i=0;i<(size-1); i++) // This loop will execute from 0 to 3
```

```
{
```

```
    small=arr[i]; // small = arr[0] = 1
```

```
    pos=i; // 0
```

```
    for(j=i+1;j<size;j++) // comparison of "1" with values placed  
                           // on index 1, 2 & 3
```

```
    {
```

```
        if(small>arr[j])
```

```
        {
```

```
            small=arr[j];
```

```
            pos=j;
```

```
        }
```

```
    }
```

Main
logic of
Selection
Sort

3 Sorting in Arrays (continued...)

```
        temp=arr[i];  
        arr[i]=small;  
        arr[pos]=temp;  
    }  
    cout<<" Sorted list is as follows:"<<endl;  
    for(i=0;i<size;i++)  
    {  
        cout<<arr[i]<<" ";  
    }  
    return 0;  
}
```

3 Sorting in Arrays (continued...)

Output

```
Enter size of the array: 4

Enter the elements
1
3
2
4

Sorted list is as follows:
1 2 3 4

-----

Process exited after 21.86 seconds with return value 0
Press any key to continue . . .
```

3 Sorting in Arrays (continued...)

Bubble Sort comparison of two adjoining values is done and exchange them if they are not in proper order.

Algorithm

for $i = L$ to U // "i" will execute from $L=0$ to $U=5$:

{ for $j = L$ to $[(U-1)-i]$ // "j" will execute from $L=0$ to $(5-1)-0$ i.e. $L=0$ to 4

{ if $arr[j] > arr[j+1]$ then

{ $temp = arr[j]$

$arr[j] = arr[j+1]$

$arr[j+1] = temp$

}

}

}

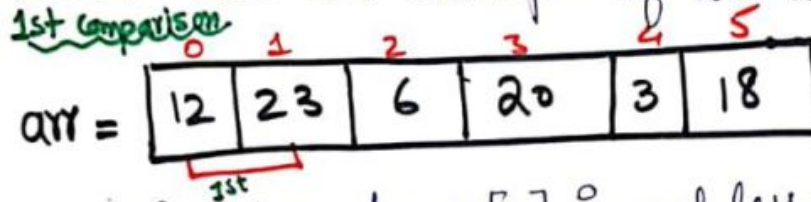
End.

3 Sorting in Arrays (continued...)

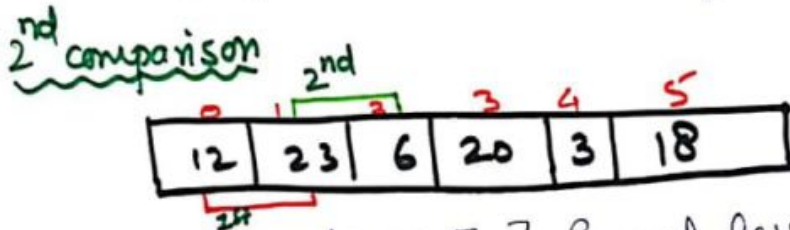
Working Example of Bubble Sort

(26)

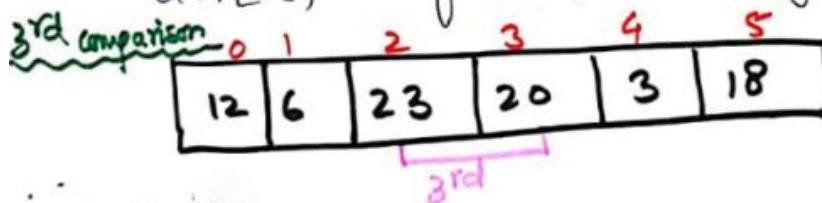
→ Let's take an example of an array



→ If value at $\text{arr}[0]$ is not less than value at $\text{arr}[1]$, then we will exchange their values.



→ The value at $\text{arr}[1]$ is not less than value at $\text{arr}[2]$, therefore we will exchange their values.



3 Sorting in Arrays (continued...)

→ The value at $\text{arr}[2]$ is not less than value at $\text{arr}[3]$, therefore we will exchange their values.

4th comparison

0	1	2	3	4	5
12	6	20	23	3	18

→ The value at $\text{arr}[3]$ is not less than value at $\text{arr}[4]$, therefore we will exchange their values.

3 Sorting in Arrays (continued...)

5th comparison

0	1	2	3	4	5
12	6	20	3	23	18

(27)

→ The value at $\text{arr}[4]$ is not less than the value at $\text{arr}[5]$, therefore we will exchange their values.

12	6	20	3	18	23
----	---	----	---	----	----

Repeat entire process again until we get the entire array sorted in ascending order i.e.,

3	6	12	18	20	23
---	---	----	----	----	----

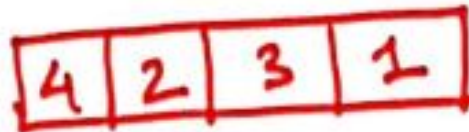
3 Sorting in Arrays (continued...)

Bubble Sort code

```
#include<iostream>

using namespace std;

int main()
{
    int arr[50], size, i, j, temp;
    cout<<"Enter the size of array:";
    cin>>size; //4
    cout<<"Enter the array elements:"<<endl;
    for(i=0;i<size;i++)
        cin>>arr[i];
```



3 Sorting in Arrays (continued...)

```
for(i=1;i<size;i++)
{
    for(j=0;j<(size-1);j++)

        if(arr[j]>arr[j+1])
        {
            temp=arr[j];
            arr[j]=arr[j+1];
            arr[j+1]=temp;
        }

    cout<<"Array after bubble sort:";

    for(i=0;i<size;i++)
        cout<<" "<<arr[i];

}

return 0;

}
```

*Main
part
or
logic of
Bubble sort*

3 Sorting in Arrays (continued...)

Output

```
Enter size of the array: 4

Enter the elements
4
2
3
1

Sorted list is as follows:
1 2 3 4

-----
Process exited after 7.51 seconds with return value 0
Press any key to continue . . .
```

3 Sorting in Arrays (continued...)

Home Work Assignment

Can two arrays be merged? If yes then write down its *algorithm*, its *working example* and its properly *running code*.

4 Multi-dimensional Arrays

- A multi-dimensional array is *an array of more than one array*.
- For example, *an array of two one-dimensional arrays* is called two-dimensional *(2D) array*.

4 Multi-dimensional Arrays (continued...)

Two Dimensional (2D) Arrays

29

- 2D array is an array where each element is itself a 1D array.
- 2D array is also called "Matrix".
- We can assume a 2D array as a table.
- If we want to access an element in a 2D array, then we must know the row & column number of that element.

2D Array representation

4 columns

00	01	02	03
10	11	12	13
20	21	22	23
30	31	32	33

4 rows

- Suppose name of array is "ar". 2D array is written as

ar[i][j]

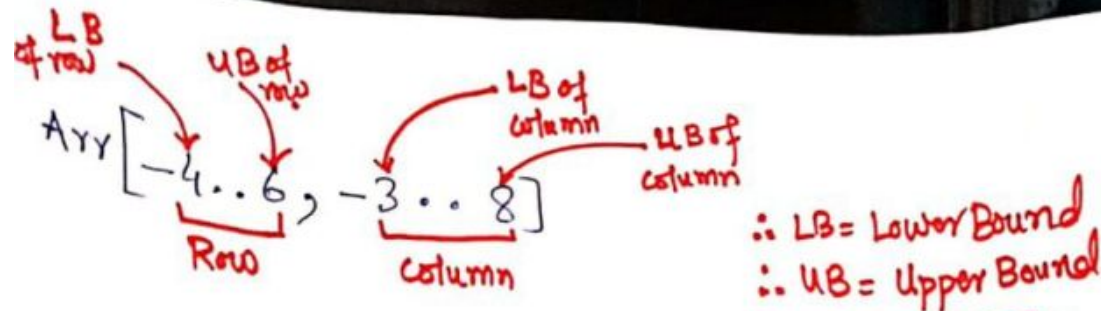
↓
row no

↑
column no

4 Multi-dimensional Arrays (continued...)

- suppose we want to access element "4" in a 2D array "ar", so the syntax to access it is: $ar[2][2] = 4$
- Total no. of elements in an array = Rows $\times$ Columns
- For example an array is given as $Arr[-4..6, -3..8]$, we have to find number of elements in this array.

4 Multi-dimensional Arrays (continued...)



30

→ Formula for calculating no of rows = $\text{UB} - \text{LB} + 1$

→ " " " " " columns = $\text{UB} - \text{LB} + 1$

→ Number of rows = $N_R = \text{UB} - \text{LB} + 1 = 6 - (-4) + 1 = 11$

→ " columns = $N_C = \text{UB} - \text{LB} + 1 = 8 - (-3) + 1 = 12$

→ Total Number of elements = $N_E = N_R \times N_C = 11 \times 12 = 132$

→ Now we will see how to implement 2D array in a memory?

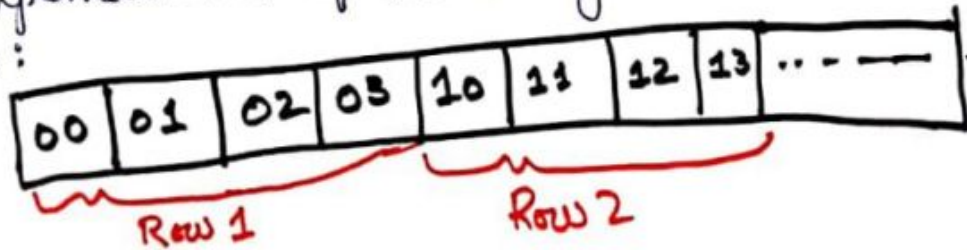
Ans: There are two methods

- i - Row-Major Implementation
- ii - Column-Major Implementation

4 Multi-dimensional Arrays (continued...)

1- Row-Major Implementation

- This technique firstly stores the first row, then it stores second row, then third row and so on.
- As memory is linear by nature, therefore row-major implementation of 2D array can be shown in memory as:



- The computer does not keep track of addresses of all the elements in the array but it keeps track of the Base address.
- Base address is the address of the very first element.

(31)

4 Multi-dimensional Arrays (continued...)

→ therefore finding address of any element in row-major implementation using formula:

$$\text{Address of } [I, J] \text{ in Row-Major order} \Rightarrow B + W[n(I - L_r) + (J - L_c)]$$

Where

- B = Base Address
- W = No. of Bytes that particular element stores or size of an element
- n = no. of columns
- I = row no. to be found
- L_r = First row no.
- J = column number to be found
- L_c = First column number

4 Multi-dimensional Arrays (continued...)

Example

$A[4..7, -1..3]$, $W = 2$ Bytes, $A[6, 2] = ?$, $B = 100$

Soln $L_r = 4$; $L_c = -1$; $I = 6$; $J = 2$

$$n = N_c = 3 - (-1) + 1 = 5$$

$$\begin{aligned} \text{Address of element} = A[6, 2] &= B + W[n(I - L_r) + (J - L_c)] \\ &= 100 + 2[5(6 - 4) + (2 - (-1))] = 126 \end{aligned}$$

4 Multi-dimensional Arrays (continued...)

(i) Column-Major Implementation

- This technique firstly stores the first column, then it stores second column, then third column & so on.
- Let's say we have a simple 2D array which has three columns and three rows as:

1	2	3

Column1	Column2	Column3
---------	---------	---------

Single Linear Memory

- Therefore we can find an address of an element in column-major implementation using formula:

$$\text{Address}[I, J] = B + W [r(J - Lc) + (I - Lr)]$$

Address of 1st Element $\leftarrow$ no. of rows.

4 Multi-dimensional Arrays (continued...)

Example

$A[-20 \dots 20, 10 \dots 35]$, $W = 1 \text{ Byte}$, $B = 500$, $A[0, 30] = ?$

Solo

Given $W = 1 \text{ Byte}$, $B = 500$, $I = 0$, $J = 30$

$Y = N_R = 20 - (-20) + 1 = 41$, $L_r = -20$, $L_c = 10$, Applying
formula, $A[0, 30] = 500 + 1[41(30 - 10) + (0 - (-20))] = 1340$

4 Multi-dimensional Arrays (continued...)

Algebra of Matrices

(i) SUM OF MATRICES

$$C[i,j] = A[i,j] + B[i,j]$$

Example

$$A \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} + B \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = C \begin{bmatrix} 5+1 & 2+6 \\ 3+7 & 4+8 \end{bmatrix} = C \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}$$

(ii) DIFFERENCE OF MATRICES

$$C[i,j] = A[i,j] - B[i,j]$$

Example

$$A = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} - B \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = C \begin{bmatrix} 5-1 & 6-2 \\ 7-3 & 8-4 \end{bmatrix} = C \begin{bmatrix} 4 & 4 \\ 4 & 4 \end{bmatrix}$$

4 Multi-dimensional Arrays (continued...)

(iii) PRODUCT OF MATRICES
 $C[i,j] = C[i,j] + A[i,k] * B[k,j]$

Example

$$\begin{matrix} a_1 \rightarrow \\ a_2 \rightarrow \end{matrix} \begin{pmatrix} 2 & 8 & 1 \\ 3 & 6 & 4 \end{pmatrix} \cdot \begin{pmatrix} b_1 & b_2 \\ 1 & 7 \\ 9 & 2 \\ 6 & 3 \end{pmatrix} = \begin{bmatrix} r_{11} & r_{12} \\ r_{21} & r_{22} \end{bmatrix} = \begin{bmatrix} 80 & 1 \\ 81 & 21 \end{bmatrix} \quad \therefore r = \text{row}$$

$$r_{11} = a_1 b_1 = (2 \ 8 \ 1) \begin{bmatrix} 1 \\ 9 \\ 6 \end{bmatrix} = 2 \times 1 + 8 \times 9 + 1 \times 6 = 80$$

$$r_{12} = a_1 b_2 = (2 \ 8 \ 1) \begin{bmatrix} 7 \\ 2 \\ 3 \end{bmatrix} = 1$$

$$r_{21} = a_2 b_1 = (3 \ 6 \ 4) \begin{bmatrix} 1 \\ 9 \\ 6 \end{bmatrix} = 81$$

$$r_{22} = a_2 b_2 = 21$$

4 Multi-dimensional Arrays (continued...)

Program: Write a program for adding two, 2x2 matrices using arrays.

```
#include<iostream>
using namespace std;
int main()
{
```

```
    int r, c, a[2][2], b[2][2], sum[2][2], i, j;
```

```
    cout<<endl<<" Enter elements of first matrix: "<<endl;
```

```
    for(i=0;i<2;i++)
```

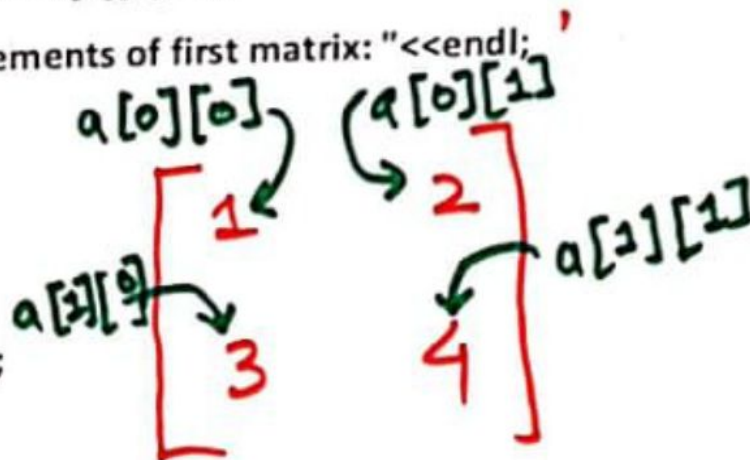
```
        for(j=0;j<2;j++)
```

```
        {
```

```
            cin>>a[i][j];
```

```
        }
```

*a[2][2] = declares a 2D array
with 2 rows & 2 columns.*



4 Multi-dimensional Arrays (continued...)

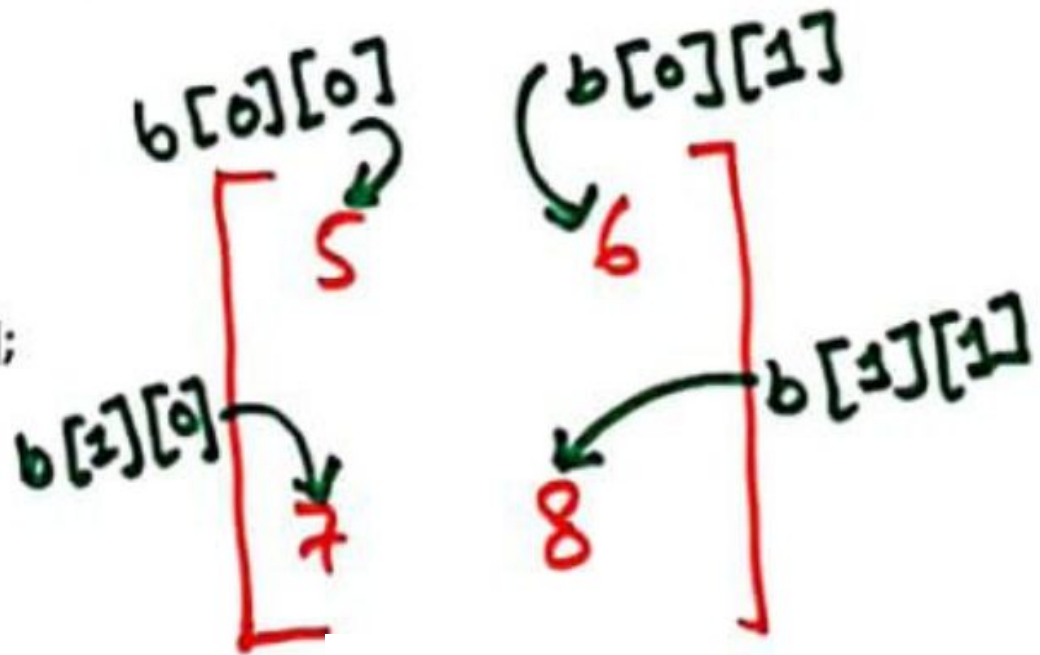
```
cout<<endl<<" Enter elements of second matrix: "<<endl;
```

```
for(i=0;i<2;i++)  
    for(j=0;j<2;j++)  
    {  
        cin>>b[i][j];  
    }
```

```
for(i=0;i<2;i++)
```

```
    for(j=0;j<2;j++)
```

```
        sum[i][j] = a[i][j] + b[i][j];
```



4 Multi-dimensional Arrays (continued...)

```
cout<<endl<<" Sum of two matrix is: "<<endl;
```

```
for(i=0;i<2;i++)
```

```
    for(j=0;j<2;j++)
```

```
    {
```

```
        cout<<sum[i][j]<<" ";
```

$Sum[i][j] =$

$$\begin{bmatrix} a[0][0] + b[0][0] & a[0][1] + b[0][1] \\ a[1][0] + b[1][0] & a[1][1] + b[1][1] \end{bmatrix}$$

$$Sum[i][j] = \begin{bmatrix} 1+5 & 2+6 \\ 3+7 & 4+8 \end{bmatrix}$$

$$Sum[i][j] = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}$$

4 Multi-dimensional Arrays (continued...)

```
        if(j==2-1)
            cout<<endl;
    }

    return 0;
}
```

4 Multi-dimensional Arrays (continued...)

Output

```
Enter elements of first matrix:
1
2
3
4

Enter elements of second matrix:
5
6
7
8

Sum of two matrix is:
6 8
10 12
```

4 Multi-dimensional Arrays (continued...)

Home Work Assignment

Program 1: Write a program for subtracting two, 2x2 matrices using arrays.

Program 2: Write a program for multiplication of two, 2x2 matrices using arrays.

5 Functions and Arrays

- An array can be passed to a function as parameter.
- When an array is passed as parameter to a function, only the address of first element of the array is passed. An array is passed by reference not by value.

Syntax:

The following syntax is used to declare a function with array as parameter:

Return-type Function-name (parameter[]);

Return-type: It indicates the type of value that will be returned by function.

Function-name: It indicates name of the function.

Return-type: Parameters are the values that are provided to a function when the function is called. The bracket with parameter indicates that the parameter will be an array.

Example: `void display(int[ ]);` // It declares a function display that will accept an array of integers as parameters.

5 Functions and Arrays (continued...)

Calling a function with Array Parameter

- A function with array parameter is called by giving the name of the array as actual parameter.
- The index of the array is not used in function call.
- The name of the array refers to the memory address of its first element.
- The memory address is passed to the function. The function then accesses the array by using the memory address.

5 Functions and Arrays (continued...)

Program: Write a program that inputs five integers in an array and passes the array to a function. The function then displays the values of the array.

Code:

```
#include<iostream>
using namespace std;
void show(int arr[]);
int main()
{
    int num[5];
    cout<<" Enter five integers: "<<endl;
    for(int i=0;i<5;i++)
    {
        cin>>num[i];
    }
    show(num);
    return 0;
}
```

5 Functions and Arrays (continued...)

```
void show(int arr[])
{
    cout<<" The values in array:"<<endl;
    for(int j=0;j<5;j++)
    {
        cout<<arr[j]<<"\t";
    }
}
```

Output

```
Enter five integers:
10
20
30
40
50
The values in array:
10      20      30      40      50
-----
Process exited after 7.363 seconds with return value 0
Press any key to continue . . .
```

References

1. *“C++ Programming: From Problem Analysis to Program Design”, 6/ed by D.S. Malik, (2013), CENGAGE Learning.*
2. *“Object Oriented Programming in C++,” 4/ed by Robert Lafore (2001) .*
3. *“C++ How to Program,” 5/ed by Deitel & Deitel (2005).*
4. *“Program with C++ Object Oriented Programming Aikman Series”, by C.M Aslam and T.A Qureshi.*
5. *Related documents from open source, mainly internet.*